



Types

▼ Type	 Lecture
📅 Date	@December 27, 2021
☰ Lecture #	2
🔗 Lecture URL	https://youtu.be/0GI9ygUk4K8
🔗 Notion URL	https://21f1003586.notion.site/Types-e3dd9b780c9d410086c44c63bffb6356
# Week #	1

The role of types

- Interpreting data stored in binary in a consistent manner
 - View sequence of bits as integers, floats, characters, ...
 - Nature and range of allowed values
 - Operations that are permitted on these values
- Naming concepts and structuring our computation
 - Especially at a higher level
 - `Point` VS `(Float, Float)`
 - Banking applications → accounts of different types, customers, ...
- Catching bugs early
 - Incorrect expression evaluation
 - Incorrect assignment

Dynamic vs Static Typing

- Every variable we use has a type
- How is the type of a variable determined
- Python determines the type based on the current value
 - **Dynamic typing** → derive type from the current value
 - `x = 10` — `x` is of type `int`
 - `x = 7.5` — now `x` is of type `float`

- An uninitialized name has no type
- **Static typing** → associate a type in advance with a name
 - Need to declare names and their types in advance
 - `int x, float a, ...`
 - Cannot assign an incompatible value — `x = 7.5` is illegal

- It is difficult to catch errors, such as typos

```
def factors(n):
    factorlist = []
    for i in range(1, n + 1):
        if n % i == 0:
            factorlst = factorlist + [i] # Typo here!
    return factorlist
```

- Empty user defined objects
 - Linked list is sequence of objects of type `Node`
 - Convenient to represent empty linked list by `None`
 - Without declaring type of `l`, Python cannot associate type after `l = None`

Types of organizing concepts

- Even simple type “synonyms” can help clarify code
 - 2D point is a pair `(float, float)`, 3D point is triple `(float, float, float)`
 - Create new type names `point2d` and `point3d`
 - These are synonyms for `(float, float)` and `(float, float, float)`
 - Makes the intent more transparent when writing, reading and maintaining code
- More elaborate types — abstract datatypes and object-oriented programming
 - Consider a banking application
 - Data and operations related to accounts, customers, deposits, withdrawals, transfers
 - Denote accounts and customers as separate types
 - Deposits, withdrawals, transfers can be applied to accounts, not to customers
 - Updating personal details applies to customers, not accounts

Static analysis

- Identify errors as early as possible — saves cost & effort
- In general, compilers cannot check that a program will work correctly
 - Halting problem — Alan Turing
- With variable declarations, compilers can detect type errors at compile time - **static analysis**
 - Dynamic typing would catch these errors only when the code runs
 - Executing code also shows down due to simultaneous monitoring for type correctness
- Compilers can also perform optimizations based on static analysis
 - Re-order statements to optimize reads and writes
 - Store previously computed expressions to re-use later

Summary

- Types have many uses
 - Making sense of arbitrary bit sequences in memory
 - Organizing concepts in our code in a meaningful way
 - Helping compilers catch bugs early, optimize compiled code

- Some languages also support automatic type inference
 - Deduce the types of variable statically, based on the context in which they are used
 - `x = 7` followed by `y = x + 15` implies `y` must be `int`
 - If the inferred type is consistent across the program, everything will go fine